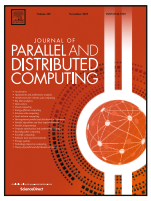
Journal of Parallel and Distributed Computing 211 (2026) 105225

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

journal homepage: www.elsevier.com/locate/jpdc



AFS-GNN: Adaptive and fast scheduling system for distributed GNN training

Yuting Gao, Yongqiang Gao *, Yongmei Liu *

School of Computer Science, Inner Mongolia University, No.235 West University Road, Hohhot, Inner Mongolia, 010021, China

ARTICLE INFO

Keywords:
Graph neural networks
Distributed training
Workload balancing
Structure-aware scheduling
Node migration

ABSTRACT

Graph Neural Networks (GNNs) have become core models for learning from relational data in domains such as transportation, social networks, and recommender systems. However, distributed GNN training on large graphs suffers from severe GPU workload imbalance and high communication cost caused by dynamic mini-batch sampling and large structural differences among nodes. To address these challenges, we propose AFS-GNN, a scheduling-aware adaptive framework that achieves fine-grained workload balancing in distributed GNN training. AFS-GNN continuously monitors per-GPU mini-batch execution time through lightweight runtime agents and employs Kalman filtering to suppress transient fluctuations and detect persistent imbalance trends. Upon imbalance detection, it constructs a Hierarchical Dependency Graph (HDG) that explicitly captures multi-hop aggregation dependencies and node-level computational costs. Guided by a heuristic load estimator, AFS-GNN applies cost-aware spectral bipartitioning via the Fiedler vector to select structurally coherent migration blocks that minimize inter-GPU communication while maintaining computational consistency. Selected blocks are migrated asynchronously across devices using intra-node or inter-process communication, ensuring non-blocking execution. Extensive experiments on large-scale benchmarks-*ogbn-products* and *ogbn-papers100M*-demonstrate that AFS-GNN achieves up to 21.7% acceleration over Euler, 15% over DistDGL, and 13.7% over FlexGraph, while maintaining stable convergence and scalability across diverse batch sizes and partition configurations.

1. Introduction

Graph neural networks (GNNs) have rapidly evolved from theoretical constructs to foundational tools across diverse domains by effectively capturing relational patterns and hierarchical dependencies in graph-structured data. For example, Google Maps leverages GNNs to model complex transportation networks, thereby significantly improving Time of Arrival (ETA) prediction accuracy [1]. In social network analysis, GNNs enable advanced community detection and influence prediction by modeling intricate user interactions [2]. Recommendation systems, meanwhile, use GNNs to capture high-order user-item relationships, achieving 15-30% accuracy gains over traditional matrix factorization methods [3]. Moreover, GNNs are increasingly applied to other challenging domains such as knowledge graphs [4], further underscoring their transformative impact on processing complex network data.

As GNNs are increasingly applied to large-scale graphs with billions of nodes and edges, efficiently training them has become a critical systems challenge. To cope with such scale, modern frameworks typically employ distributed data-parallel training combined with mini-batch sampling, improving memory efficiency and scalability [5–7]. In this paradigm, the input graph is partitioned into logical subgraphs, each assigned to a dedicated worker process. These partitions process mini-batches in parallel while periodically synchronizing model parameters across devices. However, this design introduces both structural and stochastic sources of workload imbalance. Real-world graphs often exhibit highly skewed degree distributions, and mini-batch sampling produces subgraphs with unpredictable sizes and topologies. Over time, frequently sampled or high-degree nodes tend to dominate certain partitions, resulting in persistent workload skew. This imbalance leads to synchronization delays, degraded device utilization, and increased training cost [8].

Workload imbalance is particularly pronounced in distributed GNN training with mini-batch sampling, where the computation graph dynamically changes at each training step. Although the underlying graph is static, the per-step workload distribution is highly variable. This dynamic nature leads to fluctuating per-GPU computation and communication demands, which accumulate into significant imbalance over time. Static graph partitioning methods such as METIS [9], though widely used, assume stable workload patterns and therefore struggle to

* Corresponding authors.
E-mail addresses: 1026723359@qq.com (Y. Gao), gaoyongqiang@imu.edu.cn (Y. Gao), cslym@imu.edu.cn (Y. Liu).

https://doi.org/10.1016/j.jpdc.2026.105225
Received 30 July 2025; Received in revised form 9 November 2025; Accepted 5 January 2026
Available online 8 January 2026

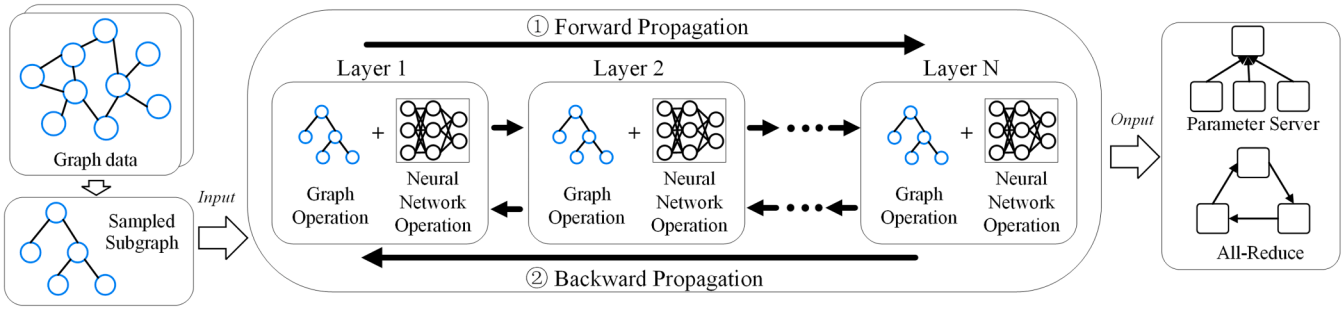


Fig. 1. Distributed GNN training workflow.

maintain balanced execution in practice. Even when reapplied periodically, these methods are inherently limited in adapting to the unpredictable sampling dynamics, resulting in sustained inefficiencies during training.

To address these limitations, recent systems have proposed adaptive strategies for mitigating workload skew. These include sampling-aware batch construction, execution configuration tuning, and dynamic task migration [8,10–13]. While such methods have shown promising improvements, they primarily focus on system-level strategies such as parameter adjustment, runtime scheduling, and delayed updates. However, they often overlook the fine-grained, structure-induced imbalance caused by the dynamic topology of sampled computation graphs. To address this issue, we adopt a structure-aware node migration strategy that explicitly accounts for topological skew and persistent imbalance across batches. Although some existing framework also perform node migration, they typically rely on instantaneous sampling or training costs to trigger decisions, making them highly sensitive to short-term fluctuations and prone to premature or ineffective migrations. For example, FlexGraph [14] estimates imbalance directly from batch-level cost signals without modeling long-term patterns, often resulting in unnecessary or mistimed migrations. Moreover, its migration policy is based on greedy selection without considering graph structure, leading to suboptimal placement and excessive cross-GPU communication. These limitations highlight the need for both more stable imbalance detection and structure-aware scheduling in distributed GNN training.

To this end, we propose AFS-GNN, a scheduling-aware framework designed to improve the efficiency and stability of distributed GNN training under dynamic and imbalanced workloads. AFS-GNN integrates lightweight runtime monitoring, adaptive imbalance detection, and structure-preserving task migration into a unified pipeline. A key feature of the system is its ability to trigger migration decisions based on workload change-point detection, enabling timely responses to runtime variability. To guide cost-effective migration, AFS-GNN quantifies node-level workloads and selectively identifies high-impact computation blocks for redistribution. These candidate subgraphs are partitioned using a spectral method that minimizes inter-GPU dependencies while matching target migration volumes. By decoupling migration planning from GPU execution and executing data transfers efficiently via intra-node communication, AFS-GNN achieves low overhead without disrupting ongoing training. Experimental results show that AFS-GNN achieves up to 21.7% speedup over Euler and 13.7% over FlexGraph, and maintains stable performance under varying batch sizes and partition counts. Ablation studies further validate the effectiveness of its workload tracker and spectral partitioning components.

Overall, AFS-GNN offers four key contributions: (1) a lightweight change point based workload monitoring mechanism for real time imbalance detection; (2) a cost guided spectral partitioning approach over Hierarchical Dependency Graphs (HDGs) to generate migration plans with minimized inter-GPU communication; (3) a structure-aware task migration design that decouples scheduling from execution enhancing responsiveness and stability; and (4) extensive experimental validation on large-scale benchmarks demonstrating reduced training latency, stability under varying scales and batch sizes, without compromising convergence accuracy, and the effectiveness of each component through ablation studies.

2. Background

This section reviews the fundamentals of GNNs and their distributed training pipeline, highlights the inefficiencies in existing systems, and analyzes critical training stages to reveal the root causes of performance bottlenecks.

2.1. Graph neural networks

Graph Neural Networks (GNNs) are a class of neural models designed for learning representations of graph-structured data. Unlike traditional neural networks that operate on grid-like structures, GNNs capture relational information by propagating and aggregating features across connected nodes. A GNN consists of multiple layers, where each layer updates the vertex features using information from neighboring nodes. The general computation in the k-th layer follows:

$$\mathbf{h}_v^{(k)} = \text{Update}^{(k)}\left(\mathbf{h}_v^{(k-1)}, \text{Aggregate}^{(k)}(\{\mathbf{h}_u^{(k-1)} | u \in \mathcal{N}(v)\})\right), \tag{1}$$

where $\mathcal{N}(v)$ denotes the neighbors of node v, and Aggregate and Update are functions that combine neighbor features and update node representations. During training, a common approach is to use mini-batches, where a set of nodes and their sampled neighbors form subgraphs. Gradient-based optimization methods, such as stochastic gradient descent, are then applied to update model parameters efficiently.

2.2. Distributed GNN training

As the scale of graph-structured data continues to grow, distributed training has become indispensable for enabling scalable and efficient Graph Neural Network (GNN) learning. A typical training pipeline, illustrated in Fig. 1, involves partitioning the input graph across machines or GPUs, generating subgraphs via neighborhood sampling (e.g., GraphSAGE [15]), and executing forward and backward propagation in a synchronized or pipelined fashion. While this paradigm resembles standard distributed deep learning, the irregular sparsity and strong inter-partition dependencies in graph data introduce distinct systems challenges.

To improve sampling efficiency, KnightKing [16] and Skywalker [17] decouple neighbor expansion from model computation to accelerate large-scale subgraph extraction. BGL [18] reuses cached neighbor data across batches to reduce redundant sampling, while LLCG [19] mitigates static-partition bias through global label correction. In terms of task scheduling and execution, GROW [20] introduces an adaptive scheduler that predicts and balances per-subgraph workload online,

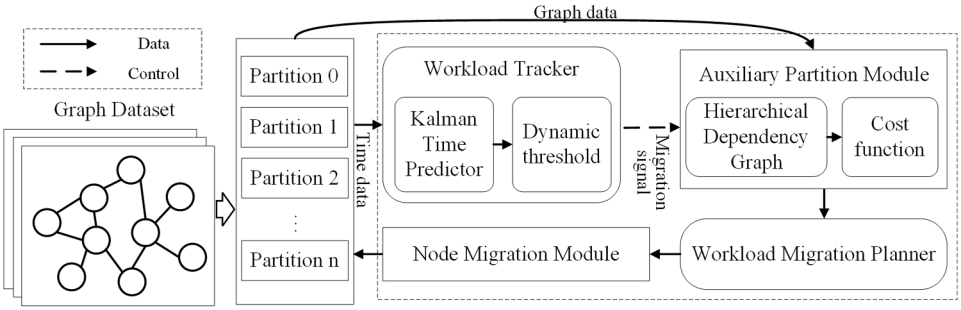


Fig. 2. Overview of AFS-GNN.

and SUBGNN [21] proposes subgraph-centric training with overlapping structures to reduce communication without hurting accuracy.

Prior work has also tackled communication and partitioning issues. ROC [22] and Cluster-GCN [23] optimize data locality by co-designing clustering with sampling. DistDGL [24] adopts multi-constraint METIS partitioning and halo node replication for static load balancing. Dorylus [25] leverages a hybrid CPU-GPU pipeline to offload shallow layers to CPU, and DGCL [26] and G3 [27] further improve training throughput with runtime-aware scheduling and structure-sensitive sampling strategies. However, these methods typically rely on static or coarse-grained partitioning, and few provide real-time monitoring or fine-grained task migration under dynamic, shifting mini-batch workloads.

3. Design overview of AFS-GNN

We therefore propose **AFS-GNN**, a dynamic and adaptive distributed GNN training system that improves training efficiency and stability under imbalanced workloads. In this section, we present an overview of AFS-GNN. Fig. 2 shows the architecture of AFS-GNN, which consists of two tightly integrated modules: a *Workload Tracker* and a *Workload Migration Planner*.

The Workload Tracker runs on each logical partition and continuously monitors the per-batch training times to capture workload variations in real time. It applies a lightweight filtering method to smooth out measurement noise and reveal underlying workload trends. When a sustained imbalance exceeding a dynamic threshold is detected, the tracker triggers a global coordination phase to balance the workload.

The Workload Migration Planner constructs a Hierarchical Dependency Graph (HDG) from the current mini-batch to model node-level computational dependencies. It assigns cost values to nodes reflecting both computation and communication demands. Aggregating these costs, the planner estimates the load distribution across partitions and identifies those with surplus workloads that require task offloading. Using a heuristic estimator, it determines the amount of workload to migrate. Then, it performs spectral bipartitioning on induced high-cost HDG subgraphs based on the Fiedler vector of the graph Laplacian. This partitioning selects migration blocks whose total cost matches the planned migration volume while minimizing cross-partition communication. Finally, the selected node blocks are asynchronously migrated to target partitions, with caches and execution contexts updated accordingly to enable uninterrupted training.

AFS-GNN works in the following steps to train a dynamic GNN model: 1) AFS-GNN continuously monitors the training time of mini-batches on each partition, smoothing temporal variations to track workload changes. 2) Upon detecting workload imbalance, it constructs the HDG for the current mini-batch and assigns cost values to nodes to estimate partition loads and determine migration volumes. 3) The planner applies spectral bipartitioning to select migration blocks from overloaded partitions that minimize communication and match planned migration volumes. 4) The selected blocks are asynchronously migrated to target logical partitions, updating local states and allowing training to proceed seamlessly.

We implement AFS-GNN atop PyTorch and DGL. Runtime profiling leverages CUDA events, with the Kalman filter in Python for portability. HDG construction and cost modeling integrate into DGL's sampling pipeline with minimal overhead. Spectral partitioning uses SciPy's sparse eigenvalue solver, and node migration is performed via direct CUDA IPC calls combined with custom synchronization primitives to enable efficient intra-node communication. This implementation relies on lightweight runtime abstractions and standard interfaces, allowing AFS-GNN to be ported to other distributed GNN frameworks such as AliGraph, DistGNN, and P3 with minimal engineering adjustments.

AFS-GNN features a modular and maintainable architecture that emphasizes both ease of integration and efficient debugging in distributed GNN systems. It is implemented as a modular layer built upon DistDGL, designed for seamless incorporation into existing training frameworks. Each component, including the Workload Tracker, the HDG Planner, and the Kalman Trend Estimator, functions as an independent module that communicates through structured runtime states and shared memory buffers. This decoupled design allows modules to be attached, replaced, or extended without altering the main training loop, ensuring smooth integration with minimal engineering effort. To further enhance transparency and maintainability, each module provides a clear interface and continuously records runtime statistics, workload trends, and migration actions in structured logs. These logs, together with clear module boundaries, enable developers to trace execution behavior, validate interactions, and tune system performance independently, facilitating reliable debugging and incremental system improvement.

4. Workload tracker

To ensure stable migration decisions, we incorporate a Kalman filter to smooth batch-level training times and suppress transient noise (Section 4.1). On top of the filtered workload signal, a time-adaptive threshold is applied during migration cost evaluation to control the sensitivity of workload redistribution (Section 4.2). This threshold gradually increases over time, discouraging migrations with diminishing returns in later training stages.

4.1. Kalman time predictor

To obtain a reliable view of partition workload evolution during distributed GNN training, we apply a Kalman filter to the per-batch execution time series. Raw batch durations L_t exhibit high-frequency noise due to sampling randomness and runtime jitter, yet also reflect long-term structural drift caused by graph topology or uneven caching. Rather than reacting to noisy signals directly, we treat L_t as a noisy observation of a latent workload state x_t, and use recursive Kalman filtering to estimate this hidden trajectory in a principled manner.

Table 1
Symbol table.

Symbol	Description
L_t	Observed workload at step t
x_t	Latent (true) workload state
$\hat{x}_{t\|t-1}$, $\hat{x}_{t\|t}$	Prior/posterior estimate of x_t before/after observing L_t
$P_{t\|t-1}$, $P_{t\|t}$	Covariance of prior/posterior estimate
Q_t, R_t	Process/observation noise variance
η_t, ϵ_t	Process/observation noise, $\sim \mathcal{N}(0, Q_t)$ or $\mathcal{N}(0, R_t)$
K_t	Kalman gain at step t
σ_t^2	Empirical variance of L_t over sliding window
α, β	Adjustment coefficients for Q_t, R_t
M_t	Trend score $\|\hat{x}_{t\|t} - \hat{x}_{t-1\|t-1}\|$
T_i	Batch time of partition i
U_T	Global average batch time
Δ_i	Relative deviation of T_i from U_T
θ_0, θ_r	Error thresholds (initial and round-specific)
η, γ	Threshold growth rate and nonlinearity parameters
f_j	Estimated cost of HDG node j
n_i, m_i	neighbors and feature size at hop i
$\mathcal{L}$	Normalized Laplacian matrix of HDG
A, D, I	Adjacency, degree, and identity matrices
v_1	Fiedler vector (2nd smallest eigenvector of $\mathcal{L}$)
$S(\tau)$, $S^c(\tau)$	Node partition by Fiedler threshold τ and its complement
$\mathrm{Ncut}(S, S^c)$	Normalized cut score between partitions

We model the partition workload at step t as a latent state variable x_t governed by a random-walk process:

$$x_t = x_{t-1} + \eta_t, \tag{2}$$

$$L_t = x_t + \epsilon_t, \tag{3}$$

where $\eta_t \sim \mathcal{N}(0, Q_t)$ captures the evolution noise of the underlying workload dynamics, and $\epsilon_t \sim \mathcal{N}(0, R_t)$ models the observation noise due to runtime measurement variance. The first equation reflects our assumption that partition workloads evolve smoothly over time in the absence of significant disruptions.

Given this model, we apply standard Kalman filtering to recursively estimate the hidden workload state from noisy runtime measurements. At each time step t, the filter performs two operations: prediction and update.

In the prediction step, we propagate the previous state estimate forward under the assumption of a random walk:

$$\hat{x}_{t|t-1} = \hat{x}_{t-1|t-1}, \tag{4}$$

$$P_{t|t-1} = P_{t-1|t-1} + Q_t, \tag{5}$$

where $\hat{x}_{t|t-1}$ is the prior estimate of the latent workload at time t, and $Pt|t-1$ is its associated uncertainty. The term Q_t captures the expected amount of process noise or drift in the workload between steps.

Upon receiving the actual runtime observation L_t, the update step adjusts the prediction using the new evidence. First, we compute the Kalman gain:

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R_t}, \tag{6}$$

which determines how much weight to assign to the new measurement. A higher K_t indicates greater reliance on the current observation, while a lower K_t favors the historical estimate.

Using K_t, we update the state estimate and its uncertainty:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(L_t - \hat{x}_{t|t-1}), \tag{7}$$

$$P_{t|t} = (1 - K_t)P_{t|t-1}. \tag{8}$$

The residual term $(L_t - \hat{x}_{t|t-1})$ represents the prediction error, which is scaled by the gain K_t to refine the estimate. The updated posterior $\hat{x}_{t|t}$ provides a smoothed approximation of the true workload at step t.

To cope with runtime variability, we estimate a short-horizon empirical variance σ_t^2 over the past w steps and adjust Q_t and R_t proportionally as:

$$Q_t = \alpha \cdot \sigma_t^2, \quad R_t = \beta \cdot \sigma_t^2, \tag{9}$$

where α and β are constant coefficients determining sensitivity to trend shifts and measurement noise. This formulation enables responsive adaptation to sudden workload changes, such as those induced by high-degree nodes, while preserving stability under normal conditions.

The filtered workload estimate $\hat{x}_{t|t}$ offers a denoised trajectory that reflects the structural trend of workload. To quantify the rate of workload change, we define a trend score M_t as:

$$M_t = |\hat{x}_{t|t} - \hat{x}_{t-1|t-1}|, \tag{10}$$

which captures the magnitude of structural variation between adjacent steps. While M_t itself does not directly trigger workload migration, its accumulation over time offers a stability signal. AFS-GNN uses this stability score to adapt migration sensitivity, suppressing unnecessary migrations under globally unstable but balanced conditions, and enhancing responsiveness during emerging imbalance phases.

4.2. Dynamic threshold

In distributed GNN training, workload migration is most beneficial during the early training stages, where imbalanced workloads can lead to significant slowdowns. As training progresses and the system gradually stabilizes, the potential performance gain from further migration diminishes, while the cost of unnecessary adjustments remains.

To reflect this cost-benefit asymmetry, we design a dynamic thresholding strategy that enforces stricter change point detection in early rounds, and becomes increasingly tolerant of fluctuations in later stages. This ensures that the system reacts to impactful workload imbalances when the optimization payoff is high, while suppressing noise-triggered rebalancing decisions when their utility becomes marginal.

In the proposed framework, workload migration is triggered when a partition's training time significantly deviates from the global execution trend. During synchronization, each partition reports its most recent batch execution time T_i, and the global average U_T is subsequently calculated to represent the overall system workload.The imbalance level of each partition is then evaluated by its relative deviation from the average:

$$\Delta_i = \frac{|T_i - U_T|}{U_T}. \tag{11}$$

A static threshold θ_0 is used to determine whether the imbalance warrants migration. Specifically, if any partition satisfies $\Delta_i > \theta_0$, the system initiates workload redistribution.

As training progresses, we gradually increase the threshold to reflect two observations: (1) the frequency of minor fluctuations tends to increase due to workload stabilization; (2) the marginal utility of triggering migrations decreases as the remaining training time shrinks.

Specifically, the threshold at training round r is defined as:

$$\theta_r = \theta_0(1 + \delta \cdot r^{\gamma}) \cdot e^{M_t}, \quad \text{with } 0 < \gamma < 1, \tag{12}$$

where δ controls the base growth rate and γ defines the degree of nonlinearity. The sublinear temporal growth ensures early responsiveness while gradually suppressing false positives in later stages. Meanwhile, the exponential modulation by the trend score M_t allows AFS-GNN to dynamically relax the threshold under unstable conditions, thereby suppressing unnecessary migrations during transient fluctuations and improving robustness.

Fig. 3 compares raw per-batch training times (grey) with Kalman-smoothed values (green). Although the average batch time is about 1.34 seconds, the original data shows significant jitter, with up to 14 percent variation between steps. Kalman filtering reduces the standard deviation

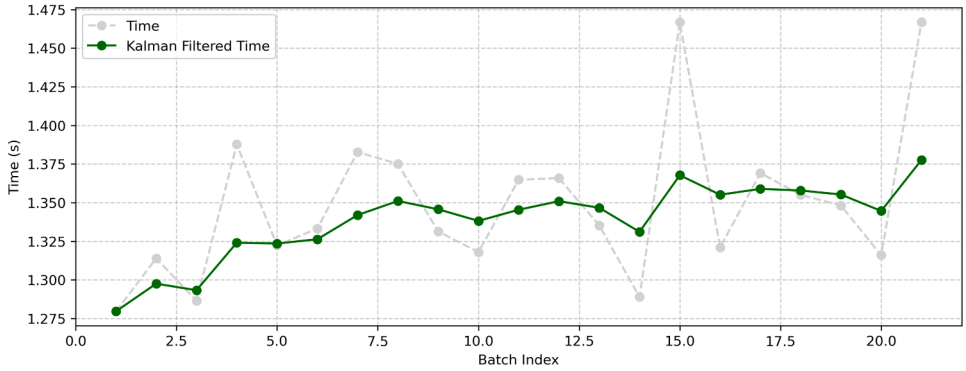


Fig. 3. Kalman-filtered batch time fluctuation.

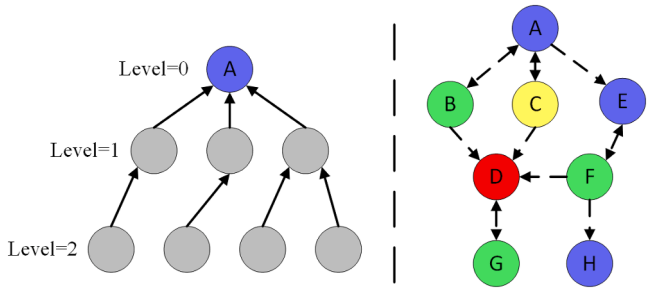


Fig. 4. HDG & induced graph.

from 0.044 to 0.015 seconds, smoothing peaks without altering overall trends. The one-step update adds negligible delay, ensuring stable estimates that prevent false imbalance alarms and improve load predictions for spectral partitioning.

5. Workload migration planner

To address runtime workload imbalance in mini-batch GNN training, AFS-GNN adopts a two-stage partitioning strategy. An auxiliary planner first estimates the migration volume for each overloaded partition and distributes it to underloaded ones based on aggregated cost metrics, without binding to specific nodes. Then, spectral partitioning selects migration blocks by decomposing cost-annotated dependency graphs, ensuring structural coherence and minimizing communication. This decoupled design enables efficient and adaptive workload balancing throughout training.

5.1. Auxiliary partition module

In mini-batch GNN training, the runtime workload across partitions may become imbalanced due to the varying size and structure of sampled computation graphs. Effective migration planning must therefore be reactive to observed workload skew, cost-aware to avoid over-migration, and structure-sensitive to preserve execution efficiency. Moreover, given the high cost of actual migration, we aim to first generate a coarse-grained migration *budget*-i.e., how much workload to migrate from each overloaded partition-without prematurely committing to specific node-level decisions. This decoupling allows later stages to focus on where and how to move the workload in a more globally coordinated manner.

Hierarchical dependency graph construction. To model fine-grained execution dependencies within each mini-batch, we construct a Hierarchical Dependency Graph (HDG) that captures how node embeddings are aggregated through the GNN's multi-hop sampling and message-passing process. The HDG is a directed acyclic graph whose root represents the seed node, internal nodes denote intermediate aggregation stages, and leaves correspond to sampled neighbors. It consists of a schema tree that encodes the aggregation hierarchy defined by the GNN model, along with a set of instantiated neighbors mapped to this schema. By merging the HDGs of all seed nodes in a mini-batch, we obtain a lightweight, batch-specific HDG that accurately reflects the actual computation and communication dependencies. This execution-aware abstraction enables precise cost modeling and supports efficient, structure-guided load balancing. Fig. 4 (left) depicts the HDG structure, highlighting its DAG nature rooted at the seed node.

Cost function. To facilitate workload-aware migration, we define a cost for each HDG node j that quantifies the computational and communication overhead induced by its multi-hop neighborhood aggregation. The cost function is modeled as a weighted sum over neighbors sampled at each hop during GNN execution:

$$f_j = \sum_{i=1}^{k} n_i \cdot m_i, \tag{13}$$

where k denotes the total number of hops (or neighbor types) considered in the aggregation, n_i represents the number of sampled neighbors at the i-th hop, and m_i is the dimensionality of their corresponding feature vectors. This formulation captures both the scale of message passing and the volume of feature aggregation operations incurred by node j, thereby reflecting its true computational load. Different from conventional graph partitioning methods that rely solely on vertex or edge counts, this cost function integrates model-specific and structure-aware factors such as sampling depth, neighborhood heterogeneity, and feature dimensionality. All parameters can be derived from runtime sampling statistics and model configurations, enabling efficient and adaptive cost estimation for load balancing.

Heuristic migration planning. Traditional load balancing methods often rely solely on static cost models or runtime measurements, which may respectively overlook dynamic performance variations or structural workload characteristics. To overcome these limitations, AFS-GNN combines two complementary signals: the HDG-based cost estimation, which captures the structural complexity of multi-hop aggregation, and the observed batch execution times, which reflect real-time partition workload and system dynamics.

Leveraging this joint perspective, we design a heuristic migration planner that estimates the aggregate workload to be reallocated between partitions. This coarse-grained budget is computed without binding to specific nodes, allowing AFS-GNN to avoid fragmented, overly frequent migrations. By decoupling migration volume estimation from fine-grained decisions, the strategy reduces coordination overhead while preserving workload balance and system stability.

5.2. Spectral graph partitioner

To enable fine-grained and structure-aware workload migration, we propose a spectral partitioning-based heuristic that leverages the cross-batch structural dependencies encoded in a cost-annotated induced graph. Unlike prior methods that perform migration on random subsets or locally selected nodes, our approach is designed to (i) preserve the semantic locality of training data, (ii) minimize inter-partition communication cost, and (iii) match the workload balancing requirements issued by the global scheduler. This is achieved by constructing a global view over multiple batches' Hierarchical Dependency Graphs (HDGs), and applying spectral decomposition techniques to identify low-cut, cost-aligned migration blocks. Our method proceeds through three key stages: induced graph generation, spectral decomposition, and partition selection.

Cost-aware induced graph construction. To enable cost-driven and structure aware migration during GNN training, we construct a cost-aware induced graph that abstracts the interdependencies among HDGs in the current mini-batch. Each mini-batch induces a Hierarchical Dependency Graph (HDG), which captures the computation flow for that batch. We treat each HDG as the minimal migration unit and construct an induced graph $G' = (V', E')$, where each node $v \in V'$ corresponds to one HDG extracted from the current batch.

To reflect both computational load and structural proximity, we annotate each node v with a cost value f_v, estimated by a cost model that considers the computation and communication cost associated with the HDG. We then establish an edge $(u, v) \in E'$ between two HDGs u and v if their constituent computation units exhibit strong structural coupling-such as sharing frontier nodes, direct dependency paths in the original HDG hierarchy, or frequent co-aggregation in sampling-based training. These edges are weighted according to a dependency strength metric (e.g., co-access frequency or shared aggregation volume), allowing the induced graph to represent both logical and workload-related interactions.

This induced graph serves as the backbone for subsequent spectral partitioning. When an overloaded partition requires migration of a target workload amount C_{mig}, we perform a spectral cut on G' to select a subset of HDG nodes whose total cost closely matches C_{mig}, while minimizing the sum of edge weights across the cut. In doing so, we avoid arbitrary migration of disconnected units and instead preserve HDG-level structural coherence during migration. This construction explicitly couples cost modeling with structural context, ensuring that the selected migration plan maintains computational contiguity and minimizes cross-partition communication overheads. The right side of the Fig. 4 shows the induced graph, where each colored node denotes a distinct HDG, and edges capture structural and computational dependencies between them.

Spectral decomposition via Fiedler approximation. To identify migration blocks that minimize inter-partition communication while respecting workload locality, we turn to spectral graph theory. We begin by constructing the normalized Laplacian matrix of the induced computation graph $G' = (V', E')$:

$$\mathcal{L} = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}, \tag{14}$$

where A denotes the adjacency matrix of G' with edge weights that reflect structural affinity, and D is the degree matrix. The second smallest eigenvector of the Laplacian, known as the Fiedler vector, provides an embedding that arranges nodes along a one-dimensional continuum. This arrangement groups strongly connected nodes together, enabling us to partition the graph effectively by cutting the ordering at a threshold. This partitioning minimizes the number of inter-partition edges, leading to reduced communication costs during migration.

Given that exact eigendecomposition of an $m \times m$ Laplacian requires $O(m^3)$ time, we use the Lanczos method to approximate the Fiedler vector. The Lanczos algorithm iteratively constructs an orthonormal basis for the Krylov subspace $\{ v, \mathcal{L}v, \mathcal{L}^2 v, \dots \}$ using a random initial vector v, yielding a tridiagonal matrix whose eigenvalues approximate those of the Laplacian. By limiting the number of iterations to $k \ll m$, we reduce the computational complexity to $O(km^2)$. To optimize performance, we warm-start Lanczos with the previously computed eigenvector and terminate early once the residual of the Rayleigh quotient.

Cut generation and workload-aligned partition selection. Once we obtain the approximate Fiedler vector v_1, we sort the induced graph nodes in ascending order of their components $v_1(i)$. Scanning this sorted list with a threshold τ induces a family of candidate bipartitions:

$$S(\tau) = \{ i \mid v_1(i) \le \tau \}, \quad S^c(\tau) = V' \setminus S(\tau). \tag{15}$$

For each split we evaluate the normalized cut

$$\text{Ncut}\left(S, S^c\right) = \frac{\text{cut}(S, S^c)}{\text{vol}(S)} + \frac{\text{cut}(S, S^c)}{\text{vol}(S^c)}, \tag{16}$$

where $\text{cut}(S, S^c)$ sums the weights of edges crossing between S and S^c, and $\text{vol}(S)$ sums node-degrees in S.

Rather than exhaustively testing every τ, we employ a greedy sweep centered at the median Fiedler value and expand outward. At each step we accept a new threshold only if it (a) strictly reduces Ncut, or (b) yields a block whose total cost

$$\left| \sum_{v \in S(\tau)} f_v - E_i \right| \le \epsilon \tag{17}$$

matches the surplus workload E_i of the source partition within tolerance ϵ. By combining structural compactness (low Ncut) with cost-alignment, we select a migration block that both minimizes communication overhead and precisely offloads the required workload.

5.3. Pipelined and asynchronous execution strategy

In large-scale GNN training, workload migration involves several nontrivial steps, including cost modeling, spectral partitioning, and state synchronization. Performing these operations synchronously often leads to "stop-the-world" delays, which degrade GPU utilization and slow down training. To address this challenge, we design a pipelined, asynchronous execution strategy that fully decouples migration planning from execution, distributing both across dedicated non-blocking threads and CUDA streams.

During training, the main training loop continuously monitors the workload distribution among partitions. When load imbalance is detected, it triggers migration requests by packaging migration plans that specify which node blocks should be moved and to which partitions. These migration requests are atomically appended to a global migration_queue, ensuring thread-safe coordination between training and migration processes.

A dedicated background thread, referred to as the MigrationWorker, asynchronously dequeues these migration plans and executes the necessary data transfers without blocking the main training path. For each migration task, the worker issues non-blocking CUDA peer-to-peer memory copies on dedicated CUDA streams, enabling concurrent data transfer and computation. Additionally, the MigrationWorker updates the partition book to reflect new node-to-partition mappings and refreshes local caches and communication buffers to maintain consistency.

To ensure correctness and avoid conflicts, all migration operations are gated by CUDA events and carefully scheduled at safe points, where the affected nodes are guaranteed to be inactive. This design prevents interference with ongoing computations, thereby maintaining high GPU utilization and training throughput.

Overall, this asynchronous and pipelined design effectively mitigates global coordination overhead by decoupling migration control from the main training flow. Instead of relying on centralized synchronization or blocking barriers, AFS-GNN achieves implicit coordination through event-driven communication and shared runtime states.

Algorithm 1 Pipelined asynchronous migration execution.

```
1: Global: lock-free queue migration_queue; per-GPU CUDA stream pool
2: function EnqueueMigrationPlan(plan)
3:     migration_queue.push(plan)
4: end function
5: function MigrationWorkerThread
6:     loop
7:         if migration_queue.not_empty() then
8:             plan ← migration_queue.pop()
9:             for all (block, target_partition) in plan do
10:                if all CUDA events on block are completed then
11:                    stream ← assign CUDA stream for target_partition
12:                    cudaMemcpyAsync(block.data, target_partition,
                       stream)
13:                    for all node ∈ block do
14:                        partition_book[node] ← target_partition
15:                    end for
16:                    invalidate local cache entries for block
17:                    prefetch remote features and adjacency lists
18:                    mark block.id as migrated
19:                else
20:                    migration_queue.push(plan)
21:                end if
22:            end for
23:        end if
24:    end loop
25: end function
```

This design allows each GPU to progress independently while maintaining system-wide consistency, ensuring that global coordination remains lightweight, scalable, and transparent to the training pipeline.

Algorithm 1 outlines our asynchronous migration engine. At each batch boundary, as defined in Lines 2 through 4, the trainer appends the migration plan to a global lock-free queue using EnqueueMigrationPlan. A dedicated MigrationWorkerThread continuously polls this queue in the background. For each block-to-partition migration pair, the worker first checks whether all associated CUDA events have completed, which corresponds to Lines 9-10, ensuring a safe point for migration. When this condition is satisfied, it assigns a dedicated CUDA stream and issues a non-blocking `cudaMemcpyAsync` to the target partition (Lines 11-13). Subsequently, the partition book is updated with new node-to-partition mappings, local caches are invalidated, and remote features and adjacency lists are prefetched (Lines 14-16). Completed blocks are then marked to prevent redundant migration. If any tensor activity is still pending, the plan is requeued for deferred execution, as shown in Line 18. This design allows migration to fully overlap with training, minimizing runtime disruption while maintaining balanced workloads.

6. Evaluation

We evaluate AFS-GNN in terms of training latency, scalability, and convergence. We also analyze its robustness to batch size variation and perform ablation studies to assess the contribution of each module.

6.1. Experimental setup

All experiments are conducted on eight NVIDIA A100 GPUs distributed across two server nodes. Within each server, GPUs communicate through high-speed PCIe interconnects, while cross-server communication is handled by 10 GbE network interfaces, which are significantly slower than intra-server PCIe links. This heterogeneous interconnect topology directly influences workload migration efficiency and overall training performance. For scalability evaluation (Section 6.5), we simulate 16 logical partitions to emulate training on a larger multi-node cluster. The initial graph is partitioned using METIS into balanced subgraphs, which are then assigned to GPU ranks before training begins. All reported training times represent the average epoch time over the first ten epochs, aggregated from multiple independent runs.

Table 2
Benchmark datasets.

Dataset	#Nodes	#Edges	Node Features
ogbn-products	2,449,029	61,859,140	100
ogbn-papers100M	111,059,956	3,231,371,744	128

We use DGL's distributed training framework with `gloo` as the backend for collective communication. Neighbor sampling, message propagation, gradient computation, and synchronization are jointly executed across ranks using PyTorch 2.4.0 and DGL 2.4.0. This setup supports runtime node migration and dynamic partition reassignment.

We evaluate three 3-layer GNN models: GraphSAGE with mean aggregation, GCN with ReLU and dropout, and GAT with multi-head attention. We use a 3-hop neighbor sampler with fan-out [10, 10, 5], and all models have a hidden size of 256 units per layer. All models use dropout (0.5) and are trained with identical neighbor sampling and batch settings.

We evaluate AFS-GNN on two large-scale node classification benchmarks from the Open Graph Benchmark suite: *ogbn-products* and *ogbn-papers100M*. The *ogbn-products* dataset contains 2.4 million nodes and 61.8 million edges, while *ogbn-papers100M* consists of 111 million nodes and 1.6 billion edges. These datasets provide diverse topologies and varying workload characteristics, enabling comprehensive evaluation of training efficiency under different scales and structures.

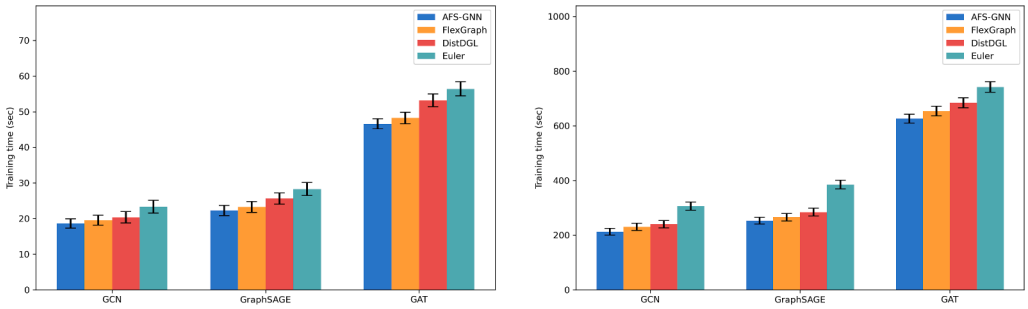
We compare AFS-GNN against three representative distributed GNN systems. DistDGL adopts static graph partitioning and fixed task placement without runtime adaptation. Euler supports distributed execution with partial scheduling capabilities but lacks structure-aware migration. For FlexGraph, we faithfully replicated the design of its ADB component, which performs dynamic node migration based on estimated workload imbalance. All baselines are evaluated under the same training pipeline and model architecture to ensure fairness in comparison.

6.2. Training latency analysis

We compare four distributed GNN frameworks: Euler, DistDGL, FlexGraph, and AFS-GNN, on GraphSAGE, GCN, and GAT using the *ogbn-products* and *ogbn-papers100M* datasets, as illustrated in Fig. 5. Euler is optimized for storage-efficient sampling but suffers from the lack of dynamic load balancing, resulting in persistent workload imbalances when neighbor sampling varies across batches. DistDGL relies on static vertex partitions, which can create GPU bottlenecks as sampling patterns shift during training. FlexGraph partially addresses this issue by migrating hierarchical dependency graphs using a breadth-first heuristic; however, its coarse-grained evaluation intervals and limited set of migration plans restrict its ability to quickly respond to rapid workload fluctuations. These framework-specific characteristics are reflected in per-batch training time distributions: Euler and DistDGL show higher variance and occasional spikes, while FlexGraph demonstrates delayed adjustments under bursty workloads.

In contrast, AFS-GNN integrates an asynchronous scheduler directly into the training loop. Workload variations are smoothed using a Kalman filter, which reduces transient spikes and prevents unnecessary migrations. The hierarchical dependency graph is partitioned spectrally to select compact migration blocks, which are then moved asynchronously across CUDA streams without halting computation. This design enables fast adaptation to dynamic workload fluctuations while minimizing communication overhead.

The benefit of these mechanisms manifests differently across model architectures. For GCN, whose dense neighborhood aggregation often creates severe load imbalance when high-degree nodes cluster on certain GPUs, AFS-GNN detects utilization divergence early and redistributes the corresponding hotspots in real time, leading to roughly a 20% reduction in batch time on *ogbn-products*. GraphSAGE, with its more predictable sampling cost, also gains from reduced cross-partition



(a) ogbn-products (b) ogbn-papers100M

Fig. 5. Training latency versus batch size for three frameworks on *ogbn-products* and *ogbn-papers100M*.

redundancy and smoother scheduling transitions. Even in GAT, where attention-weighted message passing introduces irregular computation and memory access, AFS-GNN mitigates transient spikes and stabilizes GPU utilization.

Similar trends appear on the larger *ogbn-papers100M* dataset. AFS-GNN consistently achieves lower per-batch latency and smaller runtime variance than all baselines-for instance, GCN's batch time decreases from over 300 to about 210 seconds, while the standard deviation drops from roughly 15-19 seconds to around 10. These results highlight that AFS-GNN's adaptive scheduling and asynchronous migration remain effective even under highly dynamic and large-scale training conditions. When extended to 100 training epochs across eight GPUs, the accumulated latency reduction corresponds to a total time saving of about four hours on *ogbn-papers100M* and roughly 25 minutes on *ogbn-products*, showing that the overall efficiency gain of AFS-GNN becomes substantial in realistic distributed training scenarios.

6.3. Sensitivity analysis of scheduling thresholds

To evaluate the robustness and effectiveness of our dynamic threshold scheduler, we conduct a grid search over three key parameters: the initial threshold θ_0, the growth rate δ, and the non-linearity factor γ, on the *ogbn-products* dataset with a batch size of 1000 (Table 3). Across all 27 configurations, epoch times remain within a narrow 4% range, demonstrating that the scheduler maintains stable behavior under moderate parameter variation and that overall migration overhead consistently stays low relative to total runtime Table 1.

The initial threshold θ_0 controls the scheduler's sensitivity to early workload fluctuations. Small values trigger frequent migrations on transient load spikes, slightly increasing communication time, whereas large values delay corrective action, allowing mild imbalance to persist. An intermediate setting of 0.10 balances responsiveness and overhead, initiating migrations promptly without unnecessary transfers. The growth rate δ determines how the threshold evolves during training. Too small a value causes oscillatory migrations, while a larger value filters short-term spikes but may overlook sustained drift; $\delta = 0.010$ achieves the best trade-off. Finally, the non-linearity factor γ modulates threshold scaling with imbalance magnitude, smoothing reactions to both minor and severe workload shifts. Its impact is weaker than the other parameters, yet $\gamma = 0.50$ slightly improves stability by avoiding abrupt migration Table 2 bursts.

Overall, these results suggest that scheduler performance depends more on the interaction among parameters than any individual value, reflecting a self-stabilizing design that effectively maintains balanced GPU workloads without fine-tuning each parameter separately. Moreover, the marginal gains from automatic parameter tuning are limited and are largely offset by the additional computational overhead it would introduce.

6.4. Training efficiency under mini-batch size scaling

We evaluate four distributed GNN frameworks, Euler, DistDGL, FlexGraph, and AFS-GNN, across batch sizes of 1,000, 2,000, 4,000, and 8,000. As batch size increases, all frameworks benefit from improved GPU utilization and reduced sampling variance. However, the degree of improvement differs markedly due to their ability to handle operator-level workload imbalance. In Euler and DistDGL, static vertex partitions lead to persistent skew from high-degree nodes, resulting in uneven aggregation costs even with large batches. FlexGraph mitigates this issue through periodic graph migrations, but its coarse-grained scheduling and limited migration granularity prevent timely responses to transient imbalances, producing residual spikes between 4000 and 8,000.

AFS-GNN consistently achieves the lowest training time across all batch sizes by coordinating sampling, aggregation, and communication operators in real time (Fig. 6). On *ogbn-products*, its per-batch time drops from 22.27 seconds at a batch size of 1000 to 10.43 seconds at 8,000, while the baselines remain between 23.4 and 27.8 seconds. On *ogbn-papers100M*, AFS-GNN reaches 212.6 seconds at the largest batch, compared to 224.3-264.3 seconds for others. These improvements stem from its proactive load control: static frameworks rebuild subgraphs and process sampling sequentially, creating hotspots around high-degree nodes; FlexGraph alleviates this partially via hierarchical migration, but delayed reaction leaves transient congestion unresolved. AFS-GNN, in contrast, monitors kernel utilization during neighbor aggregation and inter-GPU exchanges, triggering compact subgraph migrations once utilization drift emerges. Migration overlaps with computation, avoiding kernel stalls and minimizing communication overhead through spectrally localized transfers.

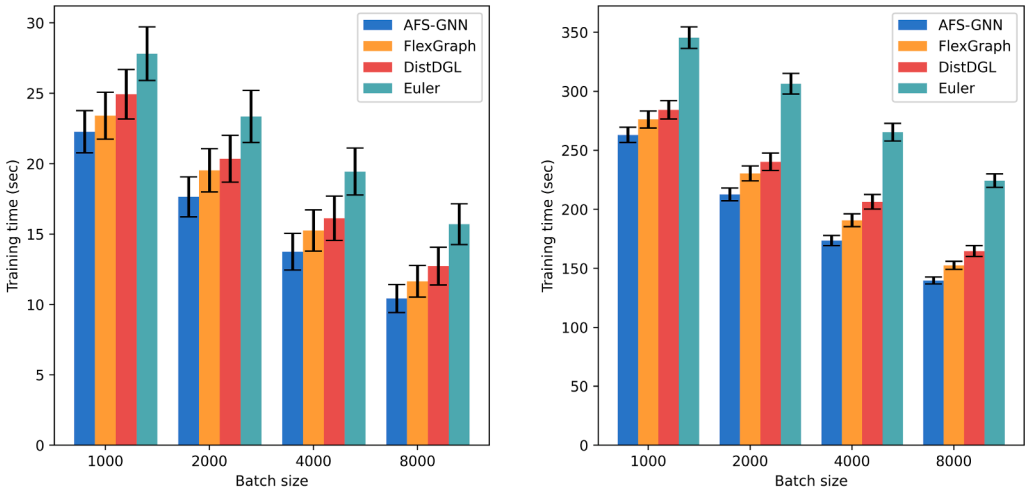
Beyond accelerating training, AFS-GNN also exhibits higher temporal stability across batches. On *ogbn-products*, its per-batch variance remains around one second, while FlexGraph and Euler fluctuate between 1.5 and 2 seconds. This stability reflects fine-grained synchronization at the operator level-when aggregation kernels begin to diverge, AFS-GNN detects early load imbalance and redistributes subgraphs before long-tail delays occur. On the larger *ogbn-papers100M* dataset, variance stays within 3 to 6 seconds, compared to 5 to 9 seconds for the baselines. The scheduler effectively filters short-term sampling bursts and only reacts to persistent imbalance, keeping GPU workloads temporally aligned. Consequently, aggregation and communication operators finish in tighter synchrony, reducing idle periods and stabilizing iteration latency under irregular sampling workloads.

6.5. Scalability under increasing partition counts

We further examine how AFS-GNN scales under increasing partition counts of 2, 4, 8, and 16 on both *ogbn-products* and *ogbn-papers100M*. As shown in Fig. 7, the performance gap between AFS-GNN and FlexGraph widens steadily with scale, growing from less than one percent at 2 GPUs to over thirteen percent at 16 GPUs. This trend reveals that while

Table 3
Threshold parameter sensitivity study with epoch times. Each group contains 3 parameter triples.

Group 1				Group 2				Group 3			
θ_0	δ	γ	Time (s)	θ_0	δ	γ	Time (s)	θ_0	δ	γ	Time (s)
0.05	0.001	0.20	28.5	0.05	0.001	0.50	27.0	0.05	0.001	0.80	28.2
0.05	0.010	0.20	26.0	0.05	0.010	0.50	25.5	0.05	0.010	0.80	26.2
0.05	0.100	0.20	25.8	0.05	0.100	0.50	25.2	0.05	0.100	0.80	25.5
0.10	0.001	0.20	27.5	0.10	0.001	0.50	26.8	0.10	0.001	0.80	26.0
0.10	0.010	0.20	25.5	0.10	0.010	0.50	**23.8**	0.10	0.010	0.80	25.6
0.10	0.100	0.20	26.0	0.10	0.100	0.50	24.5	0.10	0.100	0.80	27.0
0.20	0.001	0.20	28.0	0.20	0.001	0.50	26.5	0.20	0.001	0.80	25.5
0.20	0.010	0.20	25.8	0.20	0.010	0.50	24.2	0.20	0.010	0.80	25.0
0.20	0.100	0.20	26.5	0.20	0.100	0.50	25.8	0.20	0.100	0.80	25.2



(a) ogbn-products (b) ogbn-papers100M

Fig. 6. Training time comparison under different batch sizes.

FlexGraph's periodic migrations provide modest relief at small scales, its effectiveness diminishes as partitions multiply. The reason lies in its synchronized migration policy: every partition performs evaluation and adjustment in lockstep, which gradually fragments node placement and amplifies cross-partition dependencies. As the number of partitions increases, these fragmented boundaries inflate communication cost, and the frequent synchronization barriers delay computation progress, causing the overall training time to stagnate beyond eight GPUs.

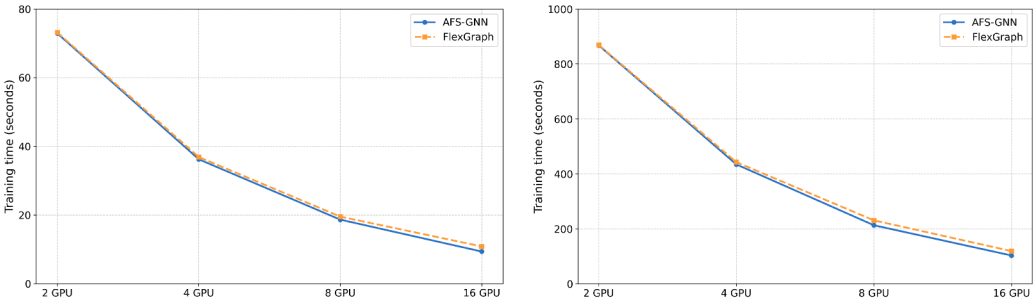
AFS-GNN exhibits a contrasting behavior. At larger scales, it not only maintains efficiency but improves relative throughput. This is because its scheduler continuously monitors both computation and communication utilization and triggers migration asynchronously, allowing each GPU to rebalance without waiting for global synchronization. When the system expands from eight to sixteen GPUs, this autonomy becomes crucial: FlexGraph's cross-partition traffic rises by roughly forty percent, while AFS-GNN restricts it to below ten percent by proactively redistributing subgraphs that contribute to heavy message flows. The spectral partitioner ensures that these migrations occur along natural community boundaries, reducing future cross-partition edges and stabilizing communication volume. Consequently, inter-GPU exchanges and aggregation kernels stay temporally aligned, preventing idle waits and sustaining near-linear scaling efficiency.

On *ogbn-papers100M*, these effects are magnified. FlexGraph's training time drops rapidly up to eight GPUs but then rebounds slightly at sixteen due to rising synchronization overhead, whereas AFS-GNN continues to improve by about thirteen percent. Kernel profiling confirms that the gap originates not from computation but from communication overlap efficiency: while FlexGraph's message-passing phase consumes over half of the total iteration time at large scales, AFS-GNN caps it below forty percent through asynchronous overlap between migration and computation. In effect, what was previously a communication bottleneck becomes a balanced, predictable component of total runtime. These results demonstrate that AFS-GNN does not merely scale better numerically-it reshapes the relationship between computation and communication, maintaining efficiency as graph partitioning deepens.

6.6. *Convergence behavior and stability*

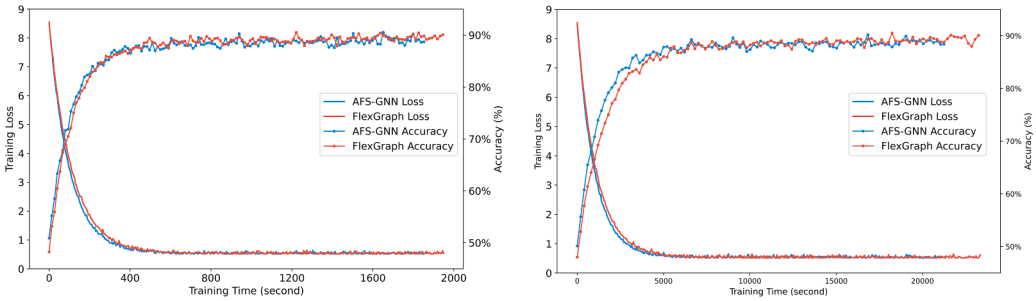
To evaluate the effectiveness of AFS-GNN in accelerating distributed GNN training, we compare its convergence trajectories against FlexGraph under a batch size of 1000. As shown in Fig. 8a, AFS-GNN reduces training loss more quickly in the early stages on ogbn-products, reaching the same final loss level in less wall-clock time. The corresponding accuracy curve rises at a similar rate and ultimately reaches the same plateau as FlexGraph, indicating that the acceleration does not compromise model performance.

A similar trend is observed on ogbn-papers100M (Fig. 8b). AFS-GNN achieves the target accuracy earlier while maintaining comparable final accuracy. Although both methods eventually converge to similar accuracy levels, AFS-GNN completes the training process in less time due to more efficient workload scheduling. These results demonstrate that AFS-GNN accelerates distributed GNN training without affecting convergence quality, making it suitable for large-scale graph learning workloads.



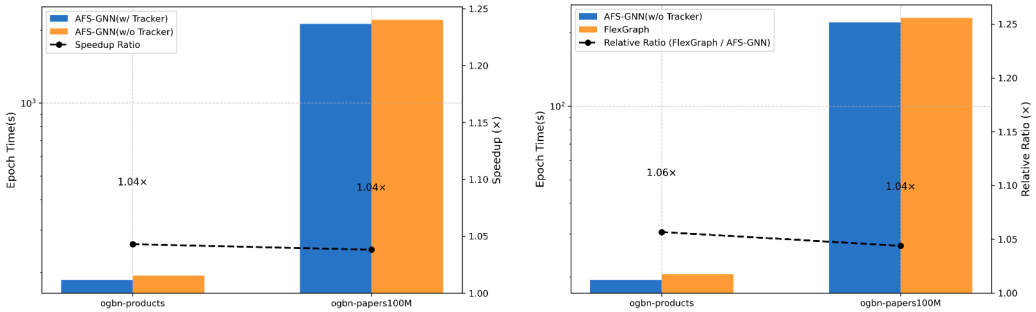
(a) ogbn-products (b) ogbn-papers100M

Fig. 7. AFS-GNN vs FlexGraph: Linear speedup w.r.t. machines.



(a) ogbn-products (b) ogbn-papers100M

Fig. 8. Training loss & validation accuracy: *AFS-GNN vs. FlexGraph*.



(a) Impact of the Tracker module (b) Comparison with FlexGraph

Fig. 9. Ablation study on workload balancing.

6.7. *Ablation experiment*

We conduct an ablation study to isolate the contribution of the Tracker module in improving workload balance. Specifically, we compare (i) the full AFS-GNN system with Tracker enabled, (ii) a baseline variant without Tracker, and (iii) the FlexGraph system.

As shown in Fig. 9, removing the Tracker module increases epoch time by 4.1% on *ogbn-products* and 3.6% on *ogbn-papers100M*, indicating its substantial contribution to load balancing and communication efficiency. Furthermore, even without the Tracker, our baseline AFS-GNN already outperforms FlexGraph by 5.3% and 4.2%, respectively. This suggests that our structure-aware scheduling and spectral partitioning provide stronger baseline capabilities than FlexGraph's greedy, BFS-based repartitioning.

FlexGraph tends to generate fragmented subgraphs and excessive cross partition communication due to its lack of topological awareness. In contrast, our system even without dynamic tracking benefits from a more informed partitioning and migration strategy. When Tracker is enabled, AFS-GNN can further adapt to runtime workload variations, refine migration planning, and maintain better structural locality, leading to reduced aggregation cost and improved GPU utilization. These results confirm that the Tracker plays a critical role in achieving robust and scalable workload balancing.

7. Discussion

Scalability and dataset characteristics. AFS-GNN is primarily designed for large-scale graphs with highly skewed degree distributions, where uneven neighbor sampling and multi-hop aggregation naturally lead to load imbalance across partitions. In contrast, smaller and denser graphs such as Reddit tend to produce more uniform workloads after partitioning, leaving limited imbalance for migration to exploit. In these cases, the per-batch computation and communication costs are both short and comparable, making migration effective yet offering only marginal overall improvement due to limited optimization headroom. Consequently, AFS-GNN yields modest speedups on such datasets while maintaining stable performance through its monitoring and Kalman-based smoothing. Future extensions could include lightweight adaptivity mechanisms,

such as dynamic migration thresholds and cost-benefit estimators, to automatically disable or delay migration when its expected gain is marginal.

System heterogeneity and topology awareness. Our current evaluation focuses on a homogeneous multi-GPU system connected via high-bandwidth PCIe and NVLink links, providing a controlled environment to isolate migration effects. However, real-world deployments often involve multi-node clusters with heterogeneous interconnects (e.g., Ethernet or InfiniBand), where communication latency and bandwidth vary across nodes. In such settings, even minor skew may cascade into synchronization delays. Future versions of AFS-GNN could incorporate topology-aware scheduling and hierarchical migration, performing fine-grained balancing within nodes before considering cross-node transfers. Integrating communication-cost models or runtime telemetry could further enable adaptive decisions under dynamic and asymmetric network conditions.

8. Conclusion

In this work, we presented AFS-GNN, a scheduling-aware framework for distributed GNN training that mitigates dynamic workload imbalance through runtime tracking, hierarchical cost modeling, and asynchronous migration. By leveraging a Hierarchical Dependency Graph and lightweight spectral partitioning, AFS-GNN achieves efficient and stable training across diverse GNNs and datasets. While its performance gain is limited on small or naturally balanced graphs, and current evaluations are confined to homogeneous multi-GPU servers, future work will explore topology-aware and adaptive scheduling to handle heterogeneous, large-scale deployments.

CRediT authorship contribution statement

Yuting Gao: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Yongqiang Gao:** Writing – review & editing, Supervision, Resources, Project administration; **Yongmei Liu:** Writing – review & editing, Supervision, Resources, Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62362054, in part by the Major Research Plan of Inner Mongolia Natural Science Foundation under Grant 2025ZD035 and in part by the Program for Young Talents of Science and Technology in Universities of Inner Mongolia Autonomous Region under Grant NJYT25007.

References

[1] A. Derrow-Pinion, J. She, D. Wong, O. Lange, T. Hester, L. Perez, M. Nunkesser, S. Lee, X. Guo, B. Wiltshire, Eta prediction with graph neural networks in google maps, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, the 30th ACM International Conference on Information & Knowledge Management, 2021, pp. 3767–3776.

[2] W. Hamilton, Z. Ying, J. Leskovec, Inductive Representation Learning on Large Graphs, 30, Curran Associates, Inc, 2017.

[3] R. Ying, R. He, K. Chen, P. Eksombatchai, W.L. Hamilton, J. Leskovec, Graph convolutional neural networks for web-scale recommender systems, in: ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2018, pp. 974–983. Proceedings of the 24th.

[4] M. Schlichtkrull, T.N. Kipf, P. Bloem, R.V. Den, I. Berg, M. Titov, Welling, Modeling relational data with graph convolutional networks, in: The Semantic Web, Cham, Springer International Publishing, 2018, pp. 593–607.

[5] M. Fey, J.E. Lenssen, Fast graph representation learning with PyTorch Geometric,2019. arXiv:1903.02428

[6] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, Deep graph library: A graph-centric, highly-performant package for graph neural networks,2019. arXiv:1909.01315

[7] R. Zhu, K. Zhao, H. Yang, W. Lin, C. Zhou, B. Ai, Y. Li, J. Zhou, AliGraph: A comprehensive graph neural network platform,2019. arXiv:1902.08730

[8] Y.C. Wan, C.R. Li, A. Wolfe, N.S. Kyrillidis, Y. Kim, Lin, PipeGCN: Efficient full-graph training of graph convolutional networks with pipelined feature communication, 2022. arxiv:2203.10428

[9] G. Karypis, V. Kumar, METIS: A Software Package For Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices, 1997.

[10] Z. Lin, C. Li, Y. Miao, Y. Liu, Y. Xu, Pagraph: scaling GNN training on large graphs via computation-aware caching, in: Proceedings of the 11th ACM Symposium on Cloud Computing, the 11th ACM Symposium on Cloud Computing, 2020, pp. 401–415.

[11] L. Ma, Z. Yang, Y. Miao, J. Xue, M. Wu, L. Zhou, Y. Dai, Neugraph: parallel deep neural network computation on large graphs, in: 2019 USENIX Annual Technical Conference (USENIX ATC 19), 2019, pp. 443–458.

[12] V. Md, S. Misra, G. Ma, R. Mohanty, E. Georganas, A. Heinecke, D. Kalamkar, N.K. Ahmed, S. Avancha, DistGNN: scalable distributed training for large-scale graph neural networks, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, the International Conference for High Performance Computing, Networking, Storage and Analysis, 2021, pp. 1–14.

[13] Y. Wang, B. Feng, G. Li, S. Li, L. Deng, Y. Xie, Y. Ding, GNNAdvisor: an adaptive and efficient runtime system for GNN acceleration on GPUs, in: 15th USENIX Symposium on Operating Systems Design and Implementation, 2021, pp. 515–531. https://www.usenix.org/conference/osdi21/presentation/wang-yuke.

[14] L. Wang, Q. Yin, J.C. Tian, R. Yang, W. Chen, Z. Yu, J. Yao, Zhou, FlexGraph: a flexible and efficient distributed framework for GNN training, in: Proceedings of the Sixteenth European Conference on Computer Systems, the Sixteenth European Conference on Computer Systems, 2021, pp. 67–82.

[15] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017).

[16] K. Yang, M. Zhang, K. Chen, X. Ma, Y. Bai, Y. Jiang, Knightking: a fast distributed graph random walk engine, in: Proceedings of the 27th ACM Symposium on Operating Systems Principles, the 27th ACM Symposium on Operating Systems Principles, 2019, pp. 524–537.

[17] P. Wang, C. Li, J. Wang, T. Wang, L. Zhang, J. Leng, Q. Chen, M. Guo, Skywalker: efficient alias-method-based graph sampling and random walk on GPUs, in: 2021 30t International Conference on Parallel Architectures and Compilation Techniques (PACT), 2021, pp. 304–317.

[18] T. Liu, Y. Chen, D. Li, C. Wu, Y. Zhu, J. He, Y. Peng, H. Chen, H. Chen, C. Guo, BGL: GPU-efficient GNN training by optimizing graph data I/O and preprocessing, 20th USENIX Symp. Netw. Syst. Des. Implement. 23 (2023) 103–118.

[19] M. Ramezani, W. Cong, M. Mahdavi, A.M.T. Kandemir, Sivasubramaniam, Learn locally, correct globally: a distributed algorithm for training graph neural networks, 2021. arXiv:2111.08202

[20] R. Hwang, M. Kang, J. Lee, D. Kam, Y. Lee, M. Rhu, GROW: a row-stationary sparse-dense GEMM accelerator for memory-efficient graph convolutional neural networks, in: 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2023, pp. 42–55.

[21] E. Alsentzer, S. Finlayson, M. Li, M. Zitnik, Subgraph neural networks, Adv. Neural Inf. Process. Syst. 33 (2020) 8017–8029.

[22] Z. Jia, S. Lin, M. Gao, M. Zaharia, A. Aiken, Improving the accuracy, scalability, and performance of graph neural networks with ROC, Proce.Mach. Learn. Syste. 2 (2020) 187–198.

[23] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, C.-J. Hsieh, Cluster-GCN: an efficient algorithm for training deep and large graph convolutional networks, in: ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 257–266. Proceedings of the 25th.

[24] D. Zheng, C. Ma, M. Wang, J. Zhou, Q. Su, X. Song, Q. Gan, Z. Zhang, G. Karypis, DistDGL: distributed graph neural network training for billion-scale graphs, in: 2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms, 2020, pp. 36–44.

[25] J. Thorpe, Y. Qiao, J. Eyolfson, S. Teng, G. Hu, Z. Jia, J. Wei, K. Vora, R. Netravali, M. Kim, Dorylus: affordable, scalable, and accurate GNN training with distributed CPU servers and serverless threads, in: 15th USENIX Symposium on Operating Systems Design and Implementation, 2021, pp. 495–514.

[26] Z. Cai, X. Yan, Y. Wu, K. Ma, J. Cheng, F. Yu, DGCL: an efficient communication library for distributed GNN training, in: Proceedings of the Sixteenth European Conference on Computer Systems, the Sixteenth European Conference on Computer Systems, 2021, pp. 130–144.

[27] H. Liu, S. Lu, X. Chen, B. He, G3: when graph neural networks meet parallel graph processing systems on GPUs, Proc. VLDB Endow. 13 (2020) 2813–2816.

Yuting Gao He received the B.S. degree in software engineering from Inner Mongolia University Of Technology, Inner Mongolia, Hohhot China, in 2023. He is currently pursuing the M.S. degree with the College of Software, Inner Mongolia University, Hohhot, China. His research interests are distributed computing, artificial intelligence, and graph neural network training.

Yongqiang Gao He received the Ph.D. degree in computer science from Shanghai Jiao Tong University, Shanghai, China, in 2013.
He is currently a Professor with the College of Computer Science, Inner Mongolia University, Hohhot, China. His research interests include cloud computing, virtualization, and green computing.

Yongmei Liu She received the M.S. degree in computer application technology from Inner Mongolia University, Inner Mongolia, Hohhot China, in 1999.
She is currently a associate professor with the College of Computer Science, Inner Mongolia University, Hohhot, China. His research interests include embedded systems and software testing engineering.